CISS240: Introduction to Programming
Quiz q1901

Name: eagraham1@cougars.ccis.edu Score:

This is a closed-book, no compiler, 5 minute quiz.

Q1. Complete the following function to check if an integer is prime. For instance `isprime(2)` returns true, `isprime(-3)` returns false, `isprime(1)` returns false, and `isprime(10)` return false,
Answer:

```
// isprime(n) return true exactly when n is a prime.
bool isprime(int n)
{
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; ++i)
    {
        if (n % i == 0) return false;
    }
    return true;
}
```

Q2. Complete the following function to compute powers. For instance `mypow(2, 0)` returns 1 and `mypow(2, 4)` returns 16.
Answer:

```
// mypow(x, y) return x to the power of y. Both x and y are integers.
// Assume y >= 0. Obviously you are not allowed to use the pow function.
// HINT: You need a loop.
int mypow(int x, int y)
{
    int result = 1;
    for (int i = 0; i < y; ++i)
    {
        result *= x;
    }
    return result;
}
```

Q3. Complete the following function that returns the number of digits in an integer. For instance `numdigits(0)` is 1, `numdigits(4030201)` is 7, `numdigits(-4030201)` is 7.

Answer:

```
int numdigits(int n)
{
    if (n < 0) n = -n;
    if (n == 0) return 1;
    int count = 0;
    while (n > 0)
    {
        n /= 10;
        count++;
    }
    return count;
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.